SIMATS ENGINEERING

Assessment Tool 2: Error Analysis Task

Course Name: Cloud Computing and Big Data Analytics | **Course Code:** CSA15
Course Outcome: CO5 - Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN (BL5)
Dave's Taxonomy: Articulation (Level 4) | **Question Count:** 5 | **Total Marks:** 25

Question 1

Q1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.

Identification of Error

The core design error is **data skew combined with poor split and partition planning**. The default Hash Partitioner is distributing keys unevenly across reducers, and/or the InputFormat is generating input splits of very unequal size (often caused by many small files or a few oversized files). As a result, a handful of mappers/reducers process most of the data while others sit idle, creating a bottleneck that can also trigger task timeouts and job failure.

Cause Analysis

The root cause is that MapReduce's default partitioning strategy assumes a roughly uniform key distribution, which rarely holds for real-world data (e.g., a few keys occurring far more frequently than others). In addition, if the source dataset has a mix of many small files and a few very large files, the default FileInputFormat creates splits proportional to block size rather than actual workload, so some mappers get far more records than others. No combiner or pre-aggregation step further amplifies the imbalance, since all raw records for a skewed key must travel to a single reducer.

Corrective Action

- Implement a **custom partitioner** that distributes skewed keys more evenly (e.g., salting/adding a random suffix to hot keys and aggregating in two stages).
- Tune split sizes using **mapreduce. input. fileinputformat. platinize/midsize** so splits are balanced relative to available mapper capacity.
- Use **CombineFileInputFormat** to merge many small files into fewer, appropriately sized splits and reduce mapper overhead.
- Introduce a **Combiner** function to perform local aggregation before the shuffle phase, reducing data sent to skewed reducers.
- Enable **speculative execution** so slow tasks caused by residual skew are backed up on other nodes.
- Where possible, pre-process/sample the dataset to identify and handle hot keys separately (e.g., iterative MapReduce or a sampling-based partitioner).

Hadoop / Integration Concepts Applied

This addresses MapReduce concepts of **partitioning, shuffle/sort, combiners, and InputFormat/split configuration**, all of which govern how work is distributed across mapper and reducer tasks in the Hadoop ecosystem.

Question 2

Q2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.

Identification of Error

The likely design error is a combination of **insufficient replication factor and/or absence of NameNode High Availability (HA)**. If the replication factor is set too low (e.g., 1) or replicas are not rack-aware, a single DataNode failure can make blocks unavailable. If the failing node is the NameNode itself, the classic single-NameNode architecture (without HA) makes the entire filesystem namespace unreachable until manual recovery.

Cause Analysis

HDFS relies on block replication (default factor 3) and rack awareness to tolerate node and rack failures; a

design that sets replication too low, or places all replicas in the same rack/node group, defeats this fault tolerance. Separately, the original HDFS architecture had a single active Name Node holding the entire namespace in memory — this is a single point of failure (SPOF). The Secondary Name Node is often mistaken for a failover node, but it only checkpoints metadata and cannot take over serving requests, so without a proper HA setup, Name Node downtime directly causes cluster-wide unavailability.

Corrective Action

- Set the **replication factor to at least 3** (the HDFS default) for critical data, adjustable via deserialization.
- Enable and correctly configure **Rack Awareness** so replicas are spread across different racks, protecting against rack-level failures.
- Deploy **HDFS High Availability (HA)** with an Active and Standby Name Node pair, using **Journal Nodes** for shared edit-log storage and a **Zookeeper Failover Controller (ZKFC)** for automatic failover.
- Use **HDFS Federation** for very large clusters to partition namespace across multiple Name Nodes and reduce blast radius of any single Name Node issue.
- Monitor Data Node health and configure appropriate **heartbeat and block-report intervals** so failures are detected and re-replication is triggered quickly.

Hadoop / Integration Concepts Applied

This applies core HDFS concepts of **block replication, rack awareness, Name Node/Data Node architecture, and High Availability (Active/Standby Name Node with Zookeeper)**, which are central to HDFS fault tolerance design.

Question 3

Q3. A data ingestion workflow using ETL and Apache Knife creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.

Identification of Error

The likely design error is the **absence of idempotent processing and deduplication logic** in a workflow that uses at-least-once delivery semantics. Knife's flow-file retry and reprocessing behaviour (e.g., after a failed connection or processor restart) can resend the same records, and if the downstream ETL step performs plain INSERTs rather than upsets, duplicates accumulate in the analytics store.

Cause Analysis

Knife guarantees at-least-once delivery by design — if a processor fails after partially delivering data but before acknowledging success, the flow file is retried, which can duplicate records that were already written. Additionally, the ETL pipeline may lack a unique key constraint or a deduplication/merge step, and provenance/tracking of already-processed records may not be enabled, so the system cannot recognize that a record has already been ingested.

Corrective Action

- Introduce **unique business keys** and use **UPSERT/MERGE** operations instead of plain INSERT when loading into the analytics store.
- Use Knife's **Detect Duplicate** processor (backed by a distributed cache) to filter out records already seen within a configurable time window.

- Enable and review the **Knife Provenance repository** to trace flow-file lineage and identify where duplication is being introduced.
- Design ingestion to be **idempotent** — e.g., include a unique record/event ID that downstream systems can use to deduplicate safely.
- Add **checkpointing/offset tracking** at ingestion boundaries so retries resume from the correct point rather than resending processed data.

Hadoop / Integration Concepts Applied

This applies concepts of **ETL pipeline design, Apache Knife flow-file semantics, at-least-once delivery, provenance tracking, and idempotent data integration** within the big data ingestion layer.

Question 4

Q4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyse the scheduling error and suggest a better resource management approach.

Identification of Error

The likely scheduling error is the use of a **FIFO Scheduler**, or a misconfigured **Capacity Scheduler** with rigid, non-elastic queues and no pre-emption. Under FIFO, large jobs submitted first monopolize cluster resources, forcing later jobs from other users to wait indefinitely (starvation) rather than sharing capacity fairly.

Cause Analysis

YARN's Resource Manager allocates containers based on the configured scheduler policy. FIFO scheduling processes jobs strictly in submission order without considering fairness across users, so one large or long-running job can consume all available containers. Even with Capacity Scheduler, if queue capacities are fixed with no elasticity and pre-emption is disabled, idle capacity in one queue cannot be reclaimed for a starved queue, worsening resource contention during concurrent multi-user submissions.

Corrective Action

- Switch to the **Fair Scheduler**, which dynamically allocates resources so all running jobs get, on average, an equal share of resources over time.
- If using **Capacity Scheduler**, configure multiple queues per user/team with appropriate minimum/maximum capacity and enable **elasticity** so idle capacity is reclaimed.
- Enable **pre-emption** so containers from over-allocated queues can be reclaimed for starved queues/users without waiting for jobs to finish.
- Apply **Dominant Resource Fairness (DRF)** to fairly allocate multiple resource types (CPU and memory) rather than a single dimension.
- Set per-user/per-queue **resource caps** (max-am-resource-percent, queue limits) to prevent any single user from monopolizing the cluster.

Hadoop / Integration Concepts Applied

This applies core YARN concepts of **Resource Manager scheduling policies (FIFO, Capacity, Fair Scheduler), queue configuration, pre-emption, and Dominant Resource Fairness** for multi-tenant resource management.

Question 5

Q5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Identification of Error

The probable design flaw is reliance on **infrequent or asynchronous backups/snapshots that are not synchronized with the live replication state**. If backups are taken on a long, fixed schedule and replication between nodes is asynchronous with lag, a restore operation pulls data from a stale snapshot or a lagging replica rather than the most recent consistent state.

Cause Analysis

Distributed storage systems often use asynchronous replication for performance, meaning replicas can briefly lag behind the primary copy. If the backup process snapshots data at fixed, infrequent intervals (rather than continuous or near-real-time capture) and does not track consistency points or version metadata, a failure occurring between backup windows results in loss of the most recent writes and restoration of outdated data. Lack of backup verification (checksums, restore testing) further allows this gap to go undetected until an actual recovery is needed.

Corrective Action

- Use HDFS's built-in **snapshot feature** more frequently, and align snapshot intervals with the system's recovery point objective (RPO).
- Where strong consistency is required, prefer **synchronous replication or write-ahead logging (WAL)** so backups capture fully committed writes.
- Use incremental backup tools like **Disc** for scheduled, efficient copying of only changed data between backup cycles.
- Implement **versioning and consistency checkpoints** so restores can be tied to a known-good, fully replicated state rather than an arbitrary snapshot.
- Regularly **test and validate backups** (checksum verification, periodic restore drills) to catch staleness or corruption before a real failure occurs.

Hadoop / Integration Concepts Applied

This applies concepts of **HDFS snapshots, replication consistency (synchronous vs asynchronous), Disc-based backup, and recovery point objectives (RPO)** in distributed storage design.
